

SCHOLAR: User Manual & Operational Guide

An AI-Powered Study Companion

1. Introduction

SCHOLAR is an advanced orchestration engine designed for engineering education. It transforms fragmented, unstructured learning materials into authoritative, syllabus-aligned "Perfect Notes" using NLP and Retrieval-Augmented Generation (RAG). The platform aims to mitigate cognitive overload by automating manual exam revision prep workflows.

2. System Requirements

To ensure optimal performance, the following hardware and software environments are recommended:

Component	Requirement
System RAM	Minimum 8GB (16GB recommended)
CPU	Multi-core modern architecture
Connectivity	Standard broadband internet
Mobile OS	Android (Kotlin-based native app)
Deployment	Docker-compatible environment for backend

3. Key Features & Functionality

3.1 Multi-Format Document Ingestion

SCHOLAR supports a wide variety of academic materials to ensure comprehensive synthesis:

- **Digital Files:** Supports multi-page PDFs and Microsoft Word files.
- **Visual Inputs:** JPG/PNG images and OCR-extracted images of handwritten notebook pages.
- **Text Assets:** Plain text and Markdown files.

3.2 AI Text Synthesis (The "Perfect Note")

The core engine uses a map-reduce pipeline for semantic deduplication.

- **Deduplication:** Automatically identifies and removes redundant content from multiple sources.
- **Syllabus Mapping:** Formats content to align directly with institutional university curricula.
- **Markdown Formatting:** Outputs structured text with clean tables and valid code highlights.

3.3 Interactive RAG Assistant

A dual-pane workspace allows users to interact with their study materials in real-time.

- **Contextual Chat:** A chat widget allows students to ask complex technical questions grounded strictly in the uploaded materials.
- **Highlight Triggers:** Highlighting any phrase in the document viewer automatically populates the chat input for instant explanation.
- **Source Citation:** The assistant provides precise file origins for its answers to ensure academic accuracy.

4. Installation & Setup

4.1 Development Environment

1. **Clone Repository:** Access the source code via the project's GitHub repository.
2. **Configuration:** Create a `.env` file containing your model and database access keys (e.g., Pinecone API Key).
3. **Launch:** Execute `docker-compose up --build` to initialize the FastAPI backend and database clusters.

4.2 Mobile Application

- The application is distributed as a signed APK or Android App Bundle (AAB).
- Users must register using institutional email credentials for secure access.

5. User Workflow

1. **Upload:** Drag and drop heterogeneous files into the target module folder.
2. **Process:** Trigger the synthesis engine to clean, deduplicate, and merge files.
3. **Study:** Review the generated "Perfect Note" in the Markdown viewer.
4. **Query:** Use the RAG assistant for context-specific tutoring on difficult concepts.

6. Limitations & Considerations

- **OCR Quality:** Legibility of handwritten notes impacts extraction accuracy; use high-contrast images where possible.
- **Token Limits:** Extremely large document sets may require aggressive chunking to stay within LLM context windows.
- **Role-Based Access:** Standard student profiles have restricted write/delete permissions; faculty profiles manage core syllabus checklists.